Software Engineering Internship

Take-Home Coding Assignment

KJBN Labs · Version 1.0 · October 2026

> **In one sentence:** take the problem you were sent (see Section 4), build a small but complete product (backend API + web/mobile UI + database), host it on a free tier, commit your progress in logical milestones, and submit a repository link, a live link, a README and a short write-up in the format given in Section 7.

1. Purpose of this assignment

We are not looking for the most features or the fanciest UI. We are looking for evidence that you can take a small business problem, model it correctly, implement its rules so they hold up under bad input, and explain your decisions clearly. Every problem below is deliberately small in domain but has enough business logic that shortcuts will show. A boring, correct, well-structured solution beats an ambitious, broken one.

What we are trying to learn about you:

- Can you translate written business rules into code and tests, including the edge cases nobody spelled out?
- Do you structure a codebase so another engineer can find things, and do you name things honestly?
- Do you handle failure deliberately — validation, exceptions, logs — or hope it never happens?
- Do you work in increments, commit meaningfully, and document as you go?
- Can you write about your own work in a precise, honest way?

2. Deliverables

#	Deliverable	Details
D1	Backend API	A JSON HTTP API implementing *all* business rules of the chosen problem. Rules must live in the backend, not only in the UI.
D2	Database	Persistent storage with a schema or migrations in the repo. SQLite is acceptable; PostgreSQL is a plus. Include a seed script that loads realistic sample data.
D3	User interface	A web UI (TypeScript) or Flutter app covering the core flows listed in the problem. It must call your API — no mocked data in the final build.
D4	Hosted deployment	Backend and UI reachable on public URLs, with seed data already loaded, for the whole review period. Free tiers that sleep and cold-start are fine — say so in the README.
D5	Deployment scripts	Everything needed to deploy from a clean clone, committed to the repo: a `Dockerfile` per service (or an equivalent build spec), a `docker-compose.yml` or single script that brings up backend + database + UI locally, the hosting provider's config file (`render.yaml`, `fly.toml`, `vercel.json`, etc.) and a migrate-and-seed script. A CI workflow that runs tests on push is strongly recommended. Manual steps done in a provider dashboard must be listed in the README.
D6	Git repository	A public GitHub/GitLab repository with a milestone-based commit history (Section 5). Monorepo or two repos — your choice; link both if two.
D7	README.md	In the exact structure given in Section 6.

#	Deliverable	Details
D8	SUBMISSION.md	Your write-up in the exact structure given in Section 7. Maximum two pages equivalent.
D9	Unit tests + coverage report	Automated unit tests for every business rule, plus API-level tests. One command runs the tests; one command (or the same one) produces a **code coverage report** for the backend. Commit the coverage summary (e.g. `coverage.txt` or the HTML index) and state the percentage in the README. Target ≥ 80% on the business-rules layer; overall percentage is reported, not graded.
D10	PROMPTS.md	A log of **every prompt** you gave to any AI assistant while building this (Section 9). Either a `PROMPTS.md` file in the repo root or a section at the end of the README.

3. Technology choices

Use a language you are productive in. We recommend the stacks below because they are what we use and review most fluently; other choices are allowed if you justify them in the README. Do **not** use a low-code builder, a CRUD generator, or a scaffolding tool that writes the business logic for you.

Layer	Recommended	Notes
Backend	**Python** (FastAPI, Django, Flask) · **Go** (net/http, chi, Gin, Echo) · **TypeScript** (Node with Express, Fastify, NestJS, Hono)	Any framework is fine. Expose OpenAPI/Swagger docs if your framework makes it easy — it helps us review.
Frontend	**TypeScript** (React, Vue, Svelte, Angular, SolidJS) · **Flutter** (web build or Android APK + web)	Plain, readable UI is enough. If you use Flutter, deploy the web build and optionally attach an APK.
Database	SQLite (file) · PostgreSQL (Neon, Supabase, Render free tier) · Turso/libSQL	Keep schema and migrations in the repo. Free Postgres tiers may pause after inactivity; document how to re-seed.
Backend hosting	Render, Fly.io, Koyeb, Google Cloud Run, Railway (trial), Deta Space, PythonAnywhere (Python only)	Free-tier terms change often — check them before you start. Containerising with a Dockerfile makes most of these trivial.
Frontend hosting	Vercel, Netlify, Cloudflare Pages, GitHub Pages, Firebase Hosting	Configure CORS on the API for your UI origin. Put the API base URL in an environment variable, not in code.

> **Non-negotiables regardless of stack:** configuration via environment variables with a committed `.env.example`; no secrets in git history; one command to run the backend, one to run the UI, one to run tests; all money values in paise/integer or decimal types — never binary floats; all timestamps stored in UTC and displayed in IST (Asia/Kolkata).

4. Your problem

You will receive **one** problem statement as a separate PDF (*Problem 1* to *Problem 5*). Each problem is a different kind of engineering challenge, calibrated to similar effort. Build only the problem you were sent; do not combine or substitute. Every problem statement has **Core** requirements (mandatory) and **Stretch** ideas (optional; attempt them only after Core is complete and tested), plus the exact edge cases reviewers will try.

Where a rule is ambiguous, make a sensible decision and record it under *Assumptions* in your README — we grade the reasoning, not whether you guessed our intent. If your problem involves time-based rules (due dates, expiry, cancellation windows), it will ask you to provide a **reviewer clock override** (e.g. an `X-Test-Clock` header or admin setting) so the logic can be tested without waiting days. Document it in README section 10.

5. Git workflow and commit guidelines

Your commit history is part of the submission and is reviewed as carefully as the code. We will run `git log --oneline --graph` and read the diff of several commits. A single "initial commit" with the whole project, or a history of "fix", "wip", "final final" messages, is an automatic deduction.

5.1 Milestones

Work through the milestones below in order. Each milestone must end with **at least one** commit whose message names the milestone (`[M2]`) and leaves the repository in a runnable state. Within a milestone, commit as often as makes sense — one logical change per commit.

Milestone	What is done when it is complete
M0 — Scaffold	Repo created, folder structure, linters/formatters configured, `.env.example`, README skeleton with the chosen problem.
M1 — Domain model	Entities and schema/migrations; seed script; the key business rules written down as test cases (failing is fine).
M2 — Business rules	Core rules implemented as pure, testable code (services/use-cases) with passing tests. No HTTP yet is acceptable.
M3 — API	Endpoints wired to the rules; validation; error responses; request logging; OpenAPI if available.
M4 — UI	Core flows working against the local API.
M5 — Deployment	Dockerfiles, compose file, provider config and migrate-and-seed script committed; backend and UI live; seed data loaded; CORS; health endpoint; CI running tests.
M6 — Documentation	README complete; SUBMISSION.md written; final tag `v1.0`.

5.2 Commit message format

Use Conventional Commits with the milestone tag in the subject. Subject ≤ 72 characters, imperative mood. The body (optional but encouraged for non-trivial commits) explains *why*, not what — the diff already shows what.

```
feat(loans): [M2] enforce 3-active-loan limit per member

Reserved, checked-out and overdue loans all count as active. Chose to
check in the service layer rather than a DB constraint because the
definition of "active" depends on the clock (overdue is derived).

fix(api): [M3] return 409 instead of 500 on double check-out
test(payout): [M2] cover advance recovery cap at 30% of gross
chore: [M0] add ruff and pre-commit config
docs: [M6] add architecture section and assumptions to README
```

Allowed types: `feat, fix, test, refactor, docs, chore, ci, perf`. Scope is the module or feature touched.

5.3 Rules

- Commit from day one; do not build locally for a week and then push once. Timestamps are visible and we look at them.
- Never commit secrets, `node_modules`, build output, or database files with real-looking data. If you accidentally do, rewrite history *and* rotate the secret.
- Do not squash the history before submitting. We want to see the path, including dead ends you backed out of (a revert commit with a reason is a positive signal).

- Branches and pull requests are optional for a solo project; if you use them, keep merge commits meaningful.
- Tag the final submission `v1.0`. Commits after the deadline are ignored.

6. README.md — required structure

Use these headings in this order. A reviewer who has never seen your project should be running it locally within five minutes using only the README. Keep it factual; marketing language is a negative signal.

Section	Must contain
1. Title and one-liner	Project name, your problem number and name, one sentence on what it does.
2. Live links	UI URL, API base URL, API docs URL (if any), test credentials if you added auth. Note cold-start delay if applicable.
3. Tech stack	Language, framework, database, hosting for each layer, and *one line on why* for any non-recommended choice.
4. Architecture	A diagram (image or ASCII) of the components and request flow; folder structure with a sentence per top-level folder; where the business rules live and why there.
5. Running locally	Prerequisites with versions; `.env` variables explained; exact commands to install, migrate, seed, run backend, run UI (and the one-command compose equivalent). Must work on a clean machine.
5a. Deployment	How the hosted instance was deployed, step by step, pointing at the scripts and config files in the repo; which steps were manual in a provider dashboard; how to redeploy after a code change; how to re-seed.
6. Running tests and coverage	The test command; the coverage command; the coverage percentage for the backend overall and for the business-rules module(s); what is covered; what is deliberately not covered and why.
7. API overview	A table of endpoints: method, path, purpose, main error codes. Link to OpenAPI if present.
8. Business rules and assumptions	Each rule from the problem, mapped to where it is enforced (file/function) and its test. Every ambiguity you resolved, as a numbered assumption.
9. Error handling and logging	Your error-response shape; how validation vs. domain vs. unexpected errors are distinguished; what is logged, at which level, with what correlation id; where logs can be seen on the hosted instance.
10. Reviewer aids	How to set the test clock; how to re-seed; sample files; anything that makes our review faster.
11. Known limitations	What does not work or was skipped, stated plainly. This section being honest earns points; it being empty costs them.
12. AI and third-party usage	Which AI assistants, templates or snippets you used and for what, and a link to your prompt log (`PROMPTS.md`, or the log itself as section 13). Allowed — but undisclosed use is disqualifying (Section 9).

7. SUBMISSION.md — your write-up (max 2 pages)

This is where you describe what you have done. It is separate from the README because its audience is different: the README is for someone running the code; this is for someone judging your engineering judgement. Use these headings exactly, in this order, and respect the length limits.

Section	Content	Limit
A. What I built	Plain-language summary of the product, which Core features are complete, which are partial.	5 sentences

Section	Content	Limit
B. Three design decisions	For each: **Decision** · **Alternative considered** · **Why I chose this** · **What it cost me**. Choose decisions where a reasonable engineer could disagree (e.g. derived vs. cached balances, where validation lives, transaction strategy).	3 × ~80 words
C. The hardest part	The specific problem that took longest or that you got wrong first; how you found out; what you changed. Reference the commit hashes.	150 words
D. How I handle failure	How a bad request, a rule violation, a database error and an unexpected exception each travel from where they occur to what the user sees and what gets logged. One short paragraph or a small table.	150 words
E. Milestone log	Table: milestone · date completed · commit hash.	7 rows
F. If I had two more days	Ordered list of what you would do next and why that order.	5 items
G. Self-assessment	Score yourself 1–5 on each rubric criterion in Section 8 with one justification each. We compare this with our scores; calibration is itself a signal.	11 rows

8. How you will be evaluated

Two engineers review every submission independently, then reconcile. Scores are out of 100 using the weights below. Shortlisted candidates get a 45-minute call where you walk us through your code and we ask you to extend it live with one small rule change — so understand every line you submit.

8.1 What we actually do

1. Open the live UI and use it as an end user for 15 minutes, deliberately trying the edge cases listed under your problem.
2. Hit the API directly with malformed input, missing fields, wrong types, out-of-range values and concurrent requests, and watch the responses and the logs.
3. Clone the repo, follow the README on a clean machine, run the tests.
4. Read the code top-down: entry point → routing → business rules → persistence. Then read the tests.
5. Read the git log and open four or five commits chosen at random plus the milestone commits.
6. Read SUBMISSION.md and compare your self-assessment with ours.

8.2 Rubric

Criterion	Wt.	Excellent (full marks) looks like	Needs work looks like
Business logic correctness	25	Every Core rule enforced in the backend; listed edge cases handled; money and time handled precisely; concurrency cases addressed and explained.	Rules only in the UI; off-by-one on dates or boundaries; float money; concurrent requests corrupt state.
Code clarity and structure	14	Clear layering (API / rules / persistence); small functions with honest names; no dead code; consistent style enforced by a linter; a newcomer can find any rule in under a minute.	Business rules inside route handlers or UI components; 300-line functions; copy-pasted blocks; misleading names.
Error and exception handling	10	Validation, domain and unexpected errors are distinct types mapped to consistent HTTP codes and one response shape; no stack traces to clients; failures leave data consistent (transactions); the UI shows actionable messages.	Everything is 500 or 200; bare `except: pass`; partial writes on failure; alerts with raw JSON.
Logging and observability	8	Structured logs with level, timestamp, request/correlation id, and the domain identifiers involved; rule violations logged at warn with context; a health endpoint; logs viewable on the hosted instance.	`print` debugging left in; secrets or PII in logs; nothing logged on errors; log noise on every request body.
Data modelling and persistence	9	Schema matches the domain; constraints and indexes where rules need them; migrations in repo; seed data realistic; derived vs. stored data chosen deliberately and documented.	Everything nullable; JSON blobs for relations; no constraints; schema created by hand on the server.
API design	8	Resource-oriented, predictable paths; correct verbs and status codes; pagination/filtering where lists can grow; OpenAPI or equivalent; versioned or at least stable shapes.	One `/doThing` endpoint; GET with side effects; inconsistent naming; unbounded lists.
UI usability	6	Core flows completable without instructions; invalid actions disabled or explained; loading and error states shown; works at laptop and phone widths.	Blank screens on error; actions available in impossible states; hard-coded API URL; only works at one window size.

Criterion	Wt.	Excellent (full marks) looks like	Needs work looks like
Testing	7	Business rules covered by fast unit tests with clear names; API-level tests; a boundary test for every numeric rule; one command runs everything; coverage report committed and ≥ 80% on the rules layer with uncovered areas explained.	Only a hello-world test; tests that hit the live server; tests that pass regardless of logic; no coverage report, or high coverage from tests without assertions.
Deployment and operability	5	Clean clone deploys with the committed scripts and documented steps; one-command local bring-up; migrate-and-seed script; health endpoint; CI runs tests; CORS and env config handled; secrets never in repo; parameterised queries and basic input sanitisation throughout.	Deployed by hand with no record of how; local setup needs undocumented steps; database created manually on the server; string-built SQL; secrets or debug endpoints exposed on the hosted instance.
Git hygiene	4	Milestone commits present; messages follow Section 5; history shows steady progress; no secrets or junk committed; tagged v1.0.	One giant commit; vague messages; secrets in history; build artefacts committed.
Documentation and write-up	4	README runs on a clean machine first time; assumptions listed; limitations honest; SUBMISSION.md follows the format, is specific, cites commits, and self-assessment matches reality.	README is the framework default; write-up is generic or exceeds limits; limitations section empty while bugs are obvious.

8.3 Automatic deductions and disqualifiers

- No working hosted link at review time: −20. (Tell us in advance if a free tier was shut down; we can re-check once.)
- No deployment scripts in the repo (deployment reproducible only from memory): −5.
- Business rules enforced only in the frontend: −15.
- Single-commit or uncommitted-until-the-last-day history: −10.
- No coverage report, or no prompt log while AI usage is declared: −5 each.
- Application crashes (5xx) on any malformed input we try: −5 per distinct crash, up to −15.
- Secrets committed to the repository: −10 and we will tell you to rotate them.
- **Disqualifying:** plagiarised code presented as your own; undisclosed AI-generated code you cannot explain on the call; submitting a different problem than the one declared in the README.

8.4 What does not matter

Visual design beyond basic clarity; number of stretch features; choice between recommended stacks; using an ORM vs. raw SQL; monorepo vs. two repos; whether you added authentication (optional; if you do, keep it simple and give us credentials).

9. Rules, AI usage and getting help

- **Timeline:** you have 7 calendar days from the email that delivered this document. Reply to that email with your links. Late submissions are reviewed only if you wrote to us before the deadline with a reason.
- **AI assistants (Copilot, ChatGPT, Claude, Cursor, etc.) are allowed.** You must disclose what you used and for what in README section 12, and you must keep a **prompt log**: every prompt you sent, in order, with the tool name and a one-line note on what you did with the answer (used as-is / edited / rejected). Put it in `PROMPTS.md` at the repo root or as the last section of the README. Chat-style autocomplete does not need logging; anything you typed as a request does.

We read the log — a thoughtful log is a positive signal, an empty one next to obviously generated code is disqualifying. You must be able to explain and modify every line on the call. We suggest using AI for boilerplate and asking it questions, and writing the business rules yourself — that is the part we are hiring for.

- **Copying** an existing project or another candidate's work is disqualifying. Using libraries is not copying.
- **Questions:** email us at any point. We answer clarification questions within one working day and share the answer with all candidates on the same problem. Decisions you make yourself and document as assumptions are never marked wrong.
- **Scope discipline:** a complete Core with zero Stretch beats a half-done Core with three Stretch items every time.
- **Data:** use invented names and phone numbers in seed data. Do not use real people.

10. Submission checklist

Reply to the assignment email with the items below. Check each one before sending.

	Item
☐	Repository URL (public) — tagged `v1.0`
☐	Live UI URL and live API URL (plus docs URL if any), with test credentials if you added auth
☐	README.md follows Section 6; a clean clone runs with the documented commands
☐	SUBMISSION.md follows Section 7 and is within limits; milestone table has real commit hashes
☐	Unit tests pass with one command; coverage report generated, committed, and percentage stated in the README
☐	Seed data is loaded on the hosted instance; the reviewer clock override (if your problem requires one) is documented
☐	No secrets, build output or dependencies in git; `.env.example` present
☐	You have personally tried every edge case listed under your problem against the hosted version
☐	AI and third-party usage disclosed; PROMPTS.md (or README prompt-log section) contains every prompt used
☐	Your name, phone and the problem number are in the email body

> Build something you would be happy to defend line by line. **— Engineering, KJBN Labs**